5/21/2026

SignLink

(FYP)

Muhammad Hassan Shahbaz
Behlol Abbas

FYP-PROPOSAL

Project Title:

SignLink

Project Description:

A Real-Time Pakistan Sign Language Translation and Subtitling into Urdu and English Speech System for Video Conferencing Platforms

Department:

Department of Computer Science & AI

Project Manager:

Dr. Muhammad Ilyas

Project Advisor:

Dr. Fahad Maqbool

Project Team:

1. Muhammad Hassan Shahbaz (BSCS51F22S007)
2. Behlol Abbas (BSCS51F22R038)

Submission Date:

7 October 2025

Supervisor's Signatures

Table of Contents

1. Abstract

The significant communication gap between the Deaf community using Pakistani Sign Language (PSL) and the hearing population in Pakistan presents a critical accessibility challenge. While global research has progressed to continuous sentence-level translation, technological solutions for PSL remain confined to isolated word recognition and alphabet fingerspelling, severely hindering natural interaction in educational, professional, and social contexts. This research proposes SignLink, the first real-time continuous PSL-to-Urdu translation system designed to process natural signing sequences and generate coherent Urdu sentences. The project aims to develop a comprehensive PSL sentence dataset through community collaboration and implement a hybrid deep learning architecture that combines spatial-temporal feature extraction with sequence-to-sequence modeling. The outcome will be a webcam-based application providing real-time translation with simultaneous Urdu text and speech output, validated through user testing with the Deaf community. The expected academic contributions include novel methodologies for continuous PSL recognition and the first publicly available PSL sentence dataset, while the industrial and social benefits encompass enhanced accessibility, reduced dependency on human interpreters, and potential applications in education and public services, ultimately fostering greater digital inclusion in Pakistani society.

2. Background and Justification

Recent years have seen growing interest in making technology more accessible. Major corporations have initiated research and development in this domain, as evidenced by the works we have studied:

Sign language recognition has evolved into a robust research domain globally, with significant progress achieved for major sign languages like American Sign Language (ASL). Modern approaches leverage deep learning architectures including Convolutional Neural Networks (CNNs) for spatial features, Recurrent Neural Networks (RNNs) for temporal dependencies, and Transformers for endto-end translation [1], [2]. The introduction of pose estimation libraries like MediaPipe has substantially advanced the field by providing robust, real-time human pose and hand landmark data [3]. Notable breakthroughs include Transformer-based frameworks for continuous sign sentence translation [4] and real-time systems using spatio-temporal Graph Convolutional Networks [5].

In stark contrast to global progress, Pakistani Sign Language research remains in its nascent stages, predominantly focused on isolated sign recognition using specialized hardware (Kinect, Leap Motion) [9], [10] or traditional computer vision techniques for limited vocabulary classification [11][16]. Comprehensive analyses consistently identify critical limitations, noting the complete absence of continuous PSL datasets and end-to-end translation models capable of processing natural signing flow [17], [18]. While some projects have developed text-to-PSL video playback systems [19], no existing work addresses the fundamental challenge of PSL-to-Urdu sentence translation.

2.1 Justification and Differentiation:

Research Gap

- **Limitation of Existing Systems**: Current PSL recognition technology is restricted to isolated alphabets and words, which is insufficient for meaningful dialogue.
- **Complexity of Continuous Translation:** The transition to sentence-level translation introduces unresolved challenges like segmenting a continuous stream of signs and modeling their nonlinear temporal dynamics.
- **Grammatical Discrepancy:** PSL has its own unique grammar that differs from spoken Urdu, a complexity that word-level systems fail to handle.

Project Justification

- **Pioneering Solution:** This project aims to develop the first real-time system for translating continuous PSL sentences into Urdu, directly addressing a critical communication barrier.
- **Technical Synthesis:** We justify our approach by building on existing PSL research while integrating advanced, proven techniques from global sign language recognition projects.
- **Practical Impact**: The primary justification is to create an accessible and practical tool that enables natural communication for the Pakistani Deaf community in social, educational, and professional settings.

3. Research Methodology

To accomplish our objectives, we will adopt the following methodology:

Phase 1: Literature Review

This is an ongoing foundational activity that informs all subsequent phases. It involves:

- **Systematic Analysis**: Critically reviewing existing research on continuous sign language recognition for global languages (ASL, BSL, CSL) to identify state-of-the-art architectures (Transformers, RNNs, GCNs, CTC loss) [1], [2], [4], [5].
- **Gap Identification:** Analyzing the specific limitations of previous Pakistani Sign Language (PSL) research, which is confined to isolated signs and hardware-dependent systems [9], [10], [11], to solidify the justification for this project.
- **Technology Selection:** Evaluating tools and libraries (e.g., MediaPipe [3], OpenCV, PyTorch , FASTAPI+websocket , Manifest V3 ,tts) based on their performance, community support, and suitability for real-time application development.
- **Ethical Framework:** Establishing guidelines for ethical data collection, informed consent, and collaboration with the Deaf community, based on best practices in assistive technology research.

Phase 2: Data Collection , Preprocessing and Dataset Development

The research will begin with the creation of the first comprehensive PSL sentence dataset through collaboration with Deaf schools and PSL interpreters. Data collection will focus on common daily communication sentences (e.g., greetings, basic questions, emergency phrases) rather than isolated signs. The methodology includes:

- Recording sentence types with multiple signers (target: 10-15 participants).

- Using standard webcams (1080p, 30fps) in consistent lighting conditions.
- Implementing MediaPipe for real-time pose and hand landmark extraction.
- Creating parallel annotations with Urdu text translations.
- Applying data augmentation techniques (rotation, scaling, brightness adjustment) to enhance model robustness.

Phase 3: Model Architecture and Development

A BiLSTM architecture will be implemented to handle the spatial-temporal nature of continuous sign language:

- **Spatial Feature Extraction:** MediaPipe will extract 3D coordinates of body, hand, and facial landmarks, reducing input dimensionality from raw video frames to structured pose .
- **Temporal Sequence Modeling**: A bidirectional LSTM architecture will process landmark sequences to capture temporal dependencies and context.
- **Sequence-to-Sequence Translation**: Connectionist Temporal Classification (CTC) loss will be employed to handle variable-length input-output sequences without explicit alignment, enabling continuous sentence translation.

Phase 4: System Integration and Application Development

A desktop application will be developed with the following components:

- Real-time Video Processing: OpenCV for webcam feed capture and preprocessing.
- Translation Engine: Integrated model inference pipeline.
- Text-to-Speech: Google gTTS or Pyttsx3 for speech synthesis.
- User Interface: Tkinter-based GUI with video display, subtitle overlay, and controls.

Phase 5: Validation and Evaluation

System performance will be evaluated through:

- **Technical Metrics**: Accuracy, precision, recall on test datasets.
- **Temporal Performance:** End-to-end latency measurement (target: <1s>.
- **User Testing**: Structured evaluation with Deaf community participants using standardized communication tasks.
- **Usability Assessment:** System usability scale (SUS) questionnaires and qualitative feedback collection.

4. Project Scope

Here are some definite scopes and out of scope things are elaborated.

In-Scope

- Integration with at least one platform (e.g., Google meet) as proof of concept.
- Functionality: Real-time sign-to-text translation (Urdu), text-to-speech conversion (English), and live subtitle display.
- Development of a continuous PSL recognition system daily use common sentence types.

- Designed for single signers in well lit, clear background environments initially.
- Real-time detection and translation of common sign languages (PSL).
- Plugin/extension for Zoom, Google Meet, etc.
- Output in English audio and Urdu onscreen subtitles.
- Support for single signer video call scenarios.

Out-of-Scope

- Support for multiple simultaneous signers.
- Translation of spoken Urdu back to PSL (reverse translation).
- Operation in poor lighting or highly cluttered environments.
- Mobile application development.
- Handling of regional PSL variations across Pakistan.

5. High level Project Plan

The project plan developed using MS Project, spans 6 months (October 2025 to March 2026) with key milestones. Below is a summarized Gantt-style overview:

Activity	Duration	Resources Assigned	Milestone/Submission Date
Literature Review & Requirements Gathering	2 weeks (Oct 1-14)	Project Manager, Team Lead	Requirements Document - Oct 14, 2025
Data Collection & Preprocessing	4 weeks (Oct 15-Nov 11)	All Group Members (4 people)	Dataset Ready - Nov 11, 2025
Model Training & Development	6 weeks (Nov 12-Dec 23)	Team Lead + 2 Members (ML expertise)	Prototype Model - Dec 23, 2025
Integration & API Development	4 weeks (Jan 1-28)	Project Manager + 2 Members (Software dev)	Integrated Module - Jan 28, 2026
Testing, Iteration & Documentation	4 weeks (Jan 29-Feb 25)	All Group Members	Final Testing Report - Feb 25, 2026
Final Presentation & Submission	2 weeks (Feb 26-May 21)	Project Manager	Complete Project - May 21, 2026

6. References

[1] O. Koller, "Advances in sign language translation and generation," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV) Workshops, 2021.

[2] O. K. et al., "Sign Language Recognition and Translation with Heterogeneous Modality," Pattern Anal. Mach. Intell., vol. 43, no. 5, 2021.

[3] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," arXiv preprint arXiv:2006.10214, 2020.

[4] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in Proc.CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020.

[5] T. Zhu, W. Li, P. Wei, and L. Wang, "Real-time continuous sign language recognition and translation with a spatial-temporal graph convolutional network," in Proc. Conf. Multimedia Expo (ICME), 2021.

[6] H. D. P. W. T. H. K. K. D. B. D. R., "American sign language recognition and translation using deep learning," in Proc. IEEE UEMCON, 2018.

[7] Y. Huang, W. Li, and L. Wang, "A method for continuous Chinese sign language recognition and translation based on key frames and attention mechanism."

[8] N. Adaloglou et al., "A comprehensive study on deep learning-based methods for sign language recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 44, no. 9, 2022.

[9] S. M. Hassan, "Pakistan Sign Language Recognition Using Kinect Sensor," in Proc. Int. Conf. Frontiers Inf. Technol. (FIT), 2017.

[10] A. R. et al., "Static Sign Word Recognition for Pakistani Sign Language Using Leap Motion Controller," in Proc. Int. Conf. Image Vis. Comput., 2019.

[11] T. Mahmood and A. I. Umar, "Urdu Sign Language Recognition Using Principal Component Analysis," in Proc. Int. Conf. Emerg. Technol. (ICET), 2018.

[12] S. Ahmed, H. Shafiq, Y. Raheel, N. Chishti, and S. M. Asad, "Pakistan sign language to Urdu translator using Kinect," Comput. Sci. Inf. Technol., vol. 3.

[13] M. Naseem, S. Sarfraz, A. Abbas, and A. Haider, "Developing a prototype to translate Pakistan sign language into text and speech while using convolutional neural networking," J. Educ. Pract., vol. 10, no. 15.

[14] H. Ahmed, S. O. Gilani, M. Jamil, Y. Ayaz, and S. I. A. Shah, "Monocular vision-based signer-independent Pakistani sign language recognition system using supervised learning," Indian J. Sci. Technol., vol. 9, no. 25, 2016.

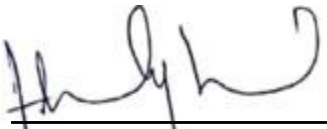
[15] A. A. Mujeeb et al., "A neural-network based web application on real-time recognition of Pakistani sign language," Eng. Appl. Artif. Intell., vol. 135, 2024.

[16] H. Zahid et al., "A computer vision-based system for recognition and classification of Urdu sign language dataset," PeerJ Comput. Sci., vol. 8, 2022.

[17] B. Hassan, M. S. Farooq, A. Abid, and N. S. Khan, "Pakistan sign language: Computer vision analysis & recommendations," VFAST Trans. Softw. Eng., vol. 4, no. 1, 2016.

[18] H. Zahid et al., "Recognition of Urdu sign language: a systematic review of the machine learning classification," PeerJ Comput. Sci., vol. 8, 2022.

[19] A. Abbas and S. Sarfraz, "Developing a prototype to translate text and speech to Pakistan sign language with bilingual subtitles: A framework," J. Educ. Technol. Syst., vol. 47, no. 2, 2018.



Supervisor's Signatures

Software Requirements Specifications

Project Title: **SignLink**

Project Description:

A Real-Time Pakistan Sign Language Translation and Subtitling into Urdu System for Video Conferencing Platforms

Project Code:

PSLTS-738

Internal Advisor:

Dr. Fahad Maqbool

Project Manager:

Dr. Muhammad Ilyas

Project Team:

Muhammad Hassan Shahbaz
Behlol Abbas

Submission Date:

20 October 2025

Supervisor's Signatures

Document Information

Category	Information
Customer	UOS
Project	SignLink
Document	Requirement Specifications
Document Version	1.2
Identifier	PSLTS-738
Status	Draft
Author(s)	M.Hassan Shabaz , Behlol Abbas
Approver(s)	Dr. Muhammad Ilyas
Issue Date	Oct. 20, 2025
Document Location	https://github.com/devHassan007/FYP-SLTS
Distribution	1. **Advisor**: Dr. Fahad Maqbool 2. **PM:** Dr. Muhammad Ilyas 3. **Project Office**: Department Of CS & AI

Definition of Terms, Acronyms and Abbreviations

Term	Description
LSTM	Long-Short Term Memory
PSL	Pakistan Sign Language
TTS	Text To Speech
SDK	Software Development Kit
ASL	American Sign Language
BSL	British Sign Language
API	Application Programming Interface
SLR	Sign Language Recognition
VSL	View Sign Language
GPU	Graphics Processing Unit

Table of Contents

1.Introduction

1.1 Purpose of Document

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the Sign Link software application. It serves as a comprehensive reference to guide the development, testing, and deployment processes, ensuring alignment with stakeholder expectations and project objectives. The document outlines the system's capabilities, constraints, and interfaces to facilitate clear communication among team members and external parties.

The intended audience includes:

1. **Project Team Members:** Developers, machine learning engineers, and testers responsible for implementing and validating the system.
2. **Project Managers and Stakeholders**: For oversight, resource allocation, and progress tracking.
3. **End Users and Representatives:** Members of the Deaf and hard-of-hearing community, accessibility experts, and potential integrators (e.g., video conferencing platform providers) to review usability and inclusivity aspects.
4. **Academic and Industrial Reviewers**: Researchers and corporate entities interested in AI-driven accessibility tools for evaluation and potential collaboration.

1.2 Project Overview

Sign Link is a software application designed to enhance accessibility in video conferencing for Deaf and hard-of-hearing individuals. It integrates with platforms like Zoom and Google Meet to provide real-time sign language recognition, translation to spoken Urdu, and live subtitling. By leveraging human pose estimation (e.g., MediaPipe) and temporal deep learning models (e.g., LSTM or Transformer) trained on datasets that will be built customly by recording videos and Multi-View Sign Language (Multi-VSL), the system extracts skeletal keypoints from video feeds, recognizes signs, and converts them into text and speech outputs.

The software will be used as a plug-and-play add-on in professional, educational, and social virtual meetings, allowing Deaf users to communicate directly without relying solely on human interpreters. Key goals include improving digital inclusivity, reducing communication barriers, and advancing real- time continuous sign language recognition. Benefits encompass academic contributions to AI accessibility, industrial applications for diverse team collaboration, and social empowerment by fostering equitable participation in remote interactions.

1.3 Scope

The project focuses on developing a prototype that enables real-time sign language translation for single users in controlled environments, with extensibility for future enhancements.

In-Scope:

1. Real-time detection and recognition of a subset of sign language vocabulary common sentences in Pakistan Sign Language (PSL).
2. Translation of recognized signs into Urdu text, followed by text-to-speech synthesis in English and subtitle display.
3. Integration as a desktop application or browser extension with at least one major video conferencing platform (e.g., Google Meet) via APIs or SDKs.
4. Support for single-signer scenarios in well-lit environments with clear backgrounds.
5. Data preprocessing, model training, and evaluation targeting >90% accuracy and <1s latency per gesture.
6. Output in English audio (injected into the audio stream) and on-screen subtitles for all participants.

Out-of-Scope:

1. Full recognition of complex sign language grammar, non-manual markers (e.g., facial expressions), or regional dialects beyond core vocabularies.
2. Simultaneous translation for multiple signers.
3. Reverse translation from spoken language to sign language.
4. Development of a standalone video conferencing platform; the focus is on an integration module.
5. Robust handling of low-light, cluttered, or noisy environments in the initial prototype.
6. Hardware-specific optimizations (e.g., for mobile or wearable devices).
7. Multilingual support beyond English outputs in the prototype phase.

2. Overall System Description

Sign Link will be developed in an academic environment using open-source tools and university computing resources, then deployed as a user-friendly application for everyday video conferencing. It anticipates users from diverse backgrounds, with constraints emphasizing accessibility and efficiency.

2.1 User characteristics

The system caters to multiple user classes, prioritized by criticality:

1. *Primary Users :*

The non-technical users are the most critical users.

- **Deaf and Hard-of-Hearing Individuals:** End users who rely on sign language for communication. They require intuitive interfaces for setup and real-time feedback, with minimal technical expertise assumed (e.g., basic computer literacy).

- **Hearing Participants in Meetings**: Non-signers who benefit from translated speech and subtitles. They expect seamless integration without disrupting workflow.

2. *Secondary Users :*

The technical users are important but less critical than primary users.

- **Human Interpreters:** Professionals who may use the tool as a supplementary aid. They possess domain knowledge in sign language and require advanced configuration options.
- **Accessibility Coordinators/Educators:** In educational or corporate settings, these users manage deployments and need administrative features for customization.

3. *Tertiary Users :*

The supportive users like developers and researchers mean a lot.

- **Developers and Researchers**: For maintenance and extensions, requiring access to source code and documentation.
- **General Public:** Casual users in social contexts, with varying technical skills.
- **User classes distinguished by needs**: Primary users focus on usability and reliability, while secondary/tertiary emphasize extensibility.

2.2 Operating environment

The software will operate on standard desktop or laptop computers, compatible with video conferencing applications. Key components include:

- **Hardware Platform:** Modern PCs with webcams, microphones, and speakers; GPU acceleration (e.g., NVIDIA CUDA-compatible) recommended for model inference.
- **Operating System:** Windows (8-11).
- **Software Components:** Python 3.x runtime; libraries such as PyTorch for ML models, OpenCV/MediaPipe for video processing, and FASTAPI Backend and Websockets for browser integration. It must coexist with video conferencing tools like Zoom or Google Meet, using virtual cameras.

2.3 System constraints

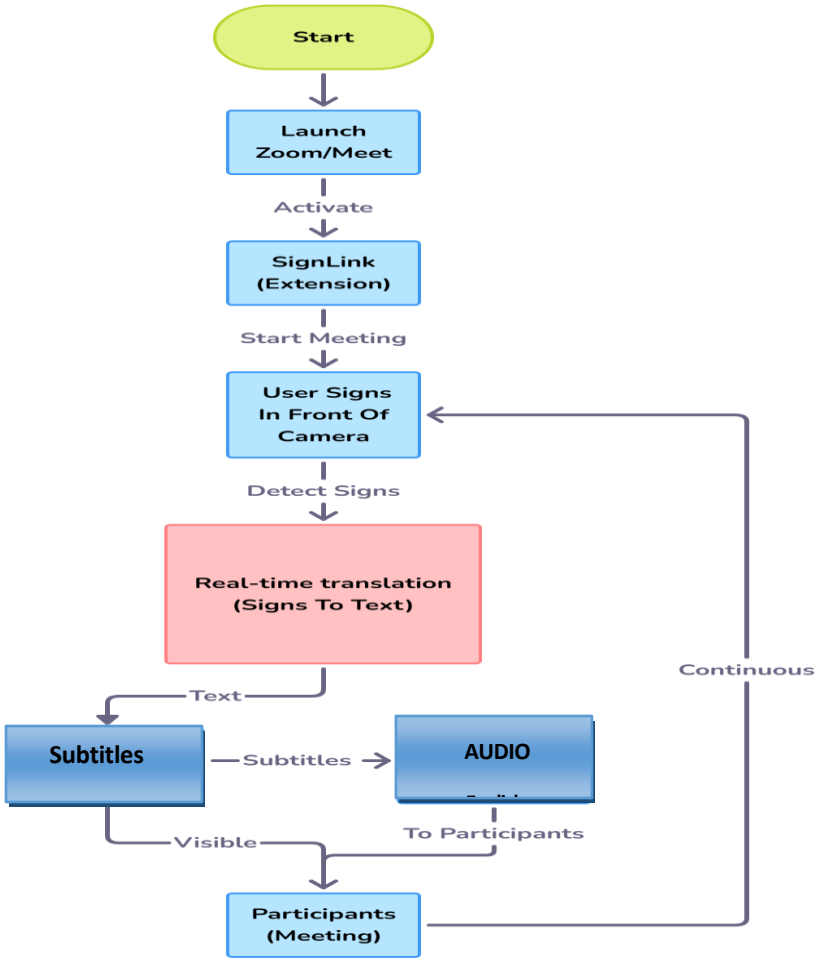
The system is subject to the following constraints:

- **Software Constraints:** Reliance on open-source libraries (e.g., MediaPipe, gTTS) limits features to available APIs; models must be optimized for real-time execution without proprietary enhancements.
- **Hardware Constraints:** Requires sufficient processing power (e.g., multi-core CPU/GPU) for low-latency inference; prototype assumes standard webcams (no specialized sensors).
- **Cultural Constraints:** Initial focus on PSL with Urdu outputs.

- **Legal Constraints:** Compliance with data privacy laws (e.g., GDPR for user video processing); privacy-preserving techniques (e.g., keypoint extraction over raw video storage) to avoid handling sensitive biometric data.
- **Environmental Constraints:** Designed for quiet, well-lit indoor settings; no audio alerts to accommodate noisy environments or hearing sensitivities.
- **User Constraints:** Interface prioritizes visual elements (e.g., graphical controls) over text for accessibility; assumes adult users but adaptable for educational contexts with simpler visuals.
- **Off-the-Shelf Components:** Datasets like PSL impose vocabulary limits; integration SDKs (e.g., Zoom API) may restrict customization, transferring any API rate limits or version dependencies to the system.

3. External Interface Requirements

The system interfaces with external components via video/audio streams and APIs. A high-level context diagram would depict: User Video Feed → Sign Link (Processing) → Video Conferencing Platform (Outputs: Subtitles/Audio), with datasets and ML libraries as supporting inputs.



3.1 Hardware Interfaces

Hardware Interfaces include the hardware components like:

1. **Webcam/Video Input:** Supports standard USB/web-integrated cameras for capturing signer video at 720p+ resolution; data interaction involves frame extraction for pose estimation.
2. **Microphone/Speaker**: Virtual audio routing for injecting synthesized speech; no direct hardware control, but compatibility with built-in or external devices.
3. **Supported Devices:** Laptops/desktops; future extensibility to mobiles, but prototype focuses on stationary setups.

3.2 Software Interfaces

Software Interfaces include components like software applications , toolset and data.

1. **Video Conferencing Platforms**: Integration with Zoom/Google Meet SDKs (name: Zoom SDK v5.x+; version: latest stable). Purpose: Hook into video streams for input and overlay subtitles/output audio. Data exchanged: Video frames (input), text subtitles (output), audio packets (synthesized speech).
2. **ML Libraries and Datasets:** MediaPipe (for pose estimation), PyTorch/TensorFlow (for models);PSL/Multi-VSL datasets (static files for training). Services: Feature extraction and inference; communications via API calls; shared data includes keypoints and model weights.
3. **TTS Engines:** Google gTTS or Pyttsx3; input: Translated text; output: Audio streams for injection.

3.3 Communications Interfaces

Communication Interfaces refer to the mediums and components that are the main concern for a communication/connection over a network.

1. **Network Protocols:** Websockets for browser-based real-time video/audio streaming; HTTP/HTTPS/WSS for API calls to external services (e.g., TTS).
2. **Message Formatting**: JSON for API payloads (e.g., subtitle overlays); synchronized timestamps for audio/subtitle alignment.
3. **Security/Encryption:** HTTPS/WSS for data transfers; no storage of raw video to ensure privacy; optional encryption for sensitive sessions.
4. **Data Transfer Rates:** Target <1s end-to-end latency; synchronization via frame buffering to handle network variability.

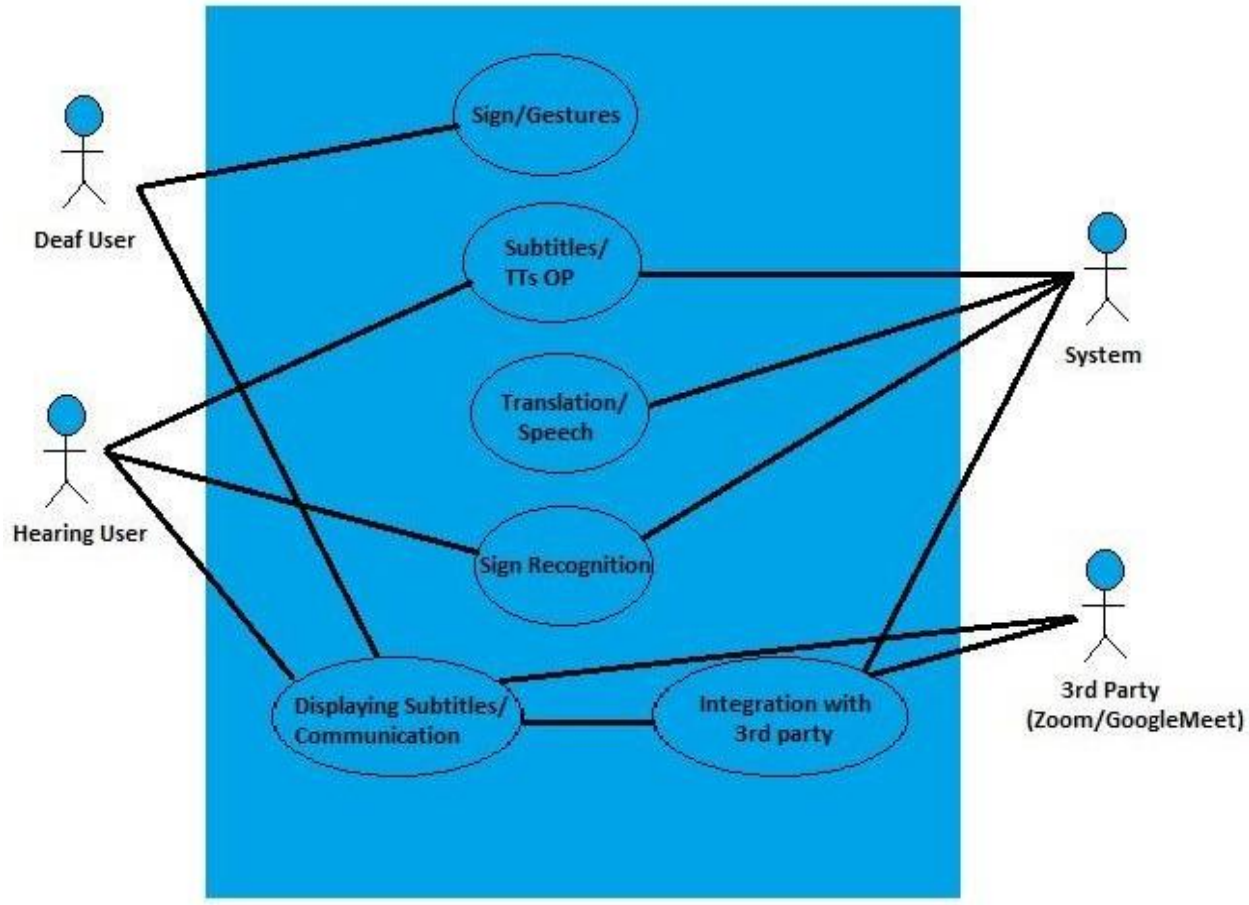
4. Functional Requirements

The system supports the following business events related to accessible video conferencing:

1. **Sign Detection and Recognition:** Capture video feed, extract keypoints, and recognize signs in real-time using trained models.
2. **Translation to Text:** Convert recognized glosses into coherent English sentences.
3. **Speech Synthesis:** Generate audio from translated text and inject into the conference audio stream.
4. **Subtitle Generation:** Display real-time text overlays for all participants.
5. **System Integration:** Hook into platform APIs for seamless input/output routing.
6. **Configuration and Setup**: Allow users to select sign language, calibrate video, and test outputs.
7. **Error Handling:** Detect low-confidence recognitions and provide fallback notifications (e.g., visual alerts).

4.1 Use Case Diagram

The use case diagram below explains two use cases discussed later.



Use Case-1

Gesture Recognition and Translation

Actors: Deaf User, System

Gesture Recognition and Translation		
Actors: Deaf User , System		
Feature: The Deaf User performs sign language gestures in front of a camera.		
Use case Id:	UC-01	
Pre-condition:	The user must have internet access and a valid device.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	Deaf User starts communication through the application.	System captures sign language input through the camera.
2.	Preprocessing and recognition occur using the trained model. .	Recognized gestures are converted to text or speech.
3.	Hearing User receives the translated message.	The system redirects the subtitles.
Alternate Scenarios:		
1a. Camera Failure → System prompts error and asks to retry.		
1b. Multiple Signers → System won't be able to translate.		
Post Conditions		
Step#	**Description**	
1.	Enable real-time understanding of sign language without a human interpreter.	
Use Case Cross referenced	UI-01: Real-time Feedback	
User Interface reference	Zoom/Google Meets/Microsoft Teams (UI-01)	
Concurrency and Response ♦ Number of concurrent users: 1 ♦ Expected response time of the use case : 1-2 sec		

Use Case-2

API Integration with Video Conferencing Platforms

Actors: System, Video Platform (Zoom/Google Meet)

API Integration with Video Conferencing Platforms		
Actors: System, Video Platform		
Feature: The system integrates with third-party video conferencing platforms through an API to enable real-time translation within online meetings.		
Use case Id:	UC-02	
Pre-condition:	The user must have internet access and Logged into third party platforms.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	The system establishes a connection with the video platform using secure API keys.	When a meeting starts, the system begins monitoring both video and audio streams.
2.	The system processes the Deaf User's gestures and the Hearing User's speech.	Recognized gestures are converted to text or speech.
3.	Translated text, speech are displayed within the video conferencing interface.	The system continues to translate until signer stops.
Alternate Scenarios:		
1a. API Failure → System prompts error and asks to retry.		
1b. Multiple Signers → System won't be able to translate.		
Post Conditions		
Step#	**Description**	
1.	This allows users to communicate seamlessly across different environments without needing a separate application.	
Use Case Cross referenced	Gesture Recognition and Translation (UC-01)	
User Interface reference	UI-002: Output Display	
Concurrency and Response ♦ Number of concurrent users: Multiple ♦ Expected response time of the use case : 1-5 sec		

Use Case-3

Model Training and Continuous Learning

Actors: System, Video Platform (Zoom/Google Meet), Database System, Training Infrastructure

Model Training and Continuous Learning		
Actors: System, Video Platform, Database System, Training Infrastructure		
Feature: System performs automatic data preprocessing. Normalizes pose landmarks for scale and rotation invariance. Applies temporal alignment to handle signing speed variations.		
Use case Id:	UC-03	
Pre-condition:	SignLink application is installed and configured. Initial training dataset has been collected.	
Scenarios		
Step#	**Action**	**Software Reaction**
1.	The administrator logs into the training dashboard and selects the dataset(s) for training.	The system preprocesses the data (normalization, augmentation, splitting) and displays the data statistics.
2.	The administrator starts the training process.	The system begins training, displaying real-time metrics (loss, accuracy) and saving checkpoints at regular intervals.
3.	The administrator validates the model and decides to deploy it.	The system packages the model and deploys it to the selected environment (staging, production, or A/B testing).
Alternate Scenarios:		
Training Data Corruption → System detects anomaly in data and halts training, prompting the administrator to verify data integrity.		
1b. Insufficient GPU Memory → System automatically reduces batch size or switches to CPU training with a warning.		
Post Conditions		
Step#	**Description**	
1.	System performs automatic data preprocessing. Normalizes pose landmarks for scale and rotation invariance. Applies temporal alignment to handle signing speed variations.	
Use Case Cross referenced	Gesture Recognition and Translation , API-Integration(UC-01 & UC-03)	
User Interface reference	None	
Concurrency and Response ♦ Parallel Training Jobs :System implements job queuing with priority levels. ♦ Simultaneous Admin Operations: Database transactions with isolation levels.		

5. Non-functional Requirements

The system doesn't support the following business events related to accessible video conferencing:

5.1 Performance Requirements

1. **Speed/Latency:** <1s per gesture recognition and translation.
2. **Accuracy/Precision:** >90% on benchmark datasets for word-level recognition.
3. **Capacity**: Handle continuous video streams up to 30 FPS; support meetings with up to 2 participants (subtitles/audio for all).
4. **Reliability:** 99% uptime in stable environments; graceful degradation in suboptimal conditions.
5. **Safety:** Minimal risk, as outputs are assistive; no critical dependencies.

5.2 Safety Requirements

The system poses low risk of harm, focusing on non-invasive video processing. Safeguards include:

- Prevention of audio feedback loops via virtual routing.
- No storage of personal video data.
- Compliance with accessibility standards) to avoid misleading translations.

No specific safety certifications required, but adherence to ethical AI guidelines for inclusivity.

5.3 Security Requirements

1. **Privacy:** Process data locally where possible; use anonymized keypoints to avoid biometric storage.
2. **Integrity:** Secure API integrations with authentication tokens.
3. **Authorization:** User-level access controls for configuration; no shared data without consent.
4. **Compliance:** Align with privacy regulations (e.g., CCPA/GDPR); no certifications mandated for prototype.

5.4 User Documentation

- **User Manual:** PDF guide covering installation, setup, and troubleshooting.
- **Online Help:** In-app tooltips and FAQs.
- **Tutorials:** Video demos for configuration and usage, accessible via the application interface.

6. Assumptions and Dependencies

Sign Link assumes stable internet and compatible hardware (e.g., webcams) while depending on external datasets, third-party libraries and video conferencing SDKs for integration and functionality.

Assumptions:

1. Availability of stable internet for platform integrations (though local processing is prioritized).
2. Users have compatible hardware (e.g., webcams); signers are in well-lit environments.
3. Datasets like PSL remain accessible and unchanged; ML models perform as benchmarked.
4. Future changes in sign language standards or platform APIs may require updates.

Dependencies:

1. Custom made datasets (PSL, Multi-VSL) for training.
2. Third-party libraries (MediaPipe, PyTorch) and their updates.
3. Video conferencing SDKs (e.g., Zoom) for integration; delays in their development could impact timelines.
4. University resources for GPU training; open-source tools for deployment.

7. References

Ref. No.	Document Title	Date of Release/ Publication	Document Source
SLTS-S07-R38	Project Proposal	Nov 30, 2025	https://github.com/devHassan007/FYP-SLTS

Software Design Specification

Project Title:

SignLink

Project Description:

A Real-Time Pakistan Sign Language Translation and Subtitling into Urdu System for Video Conferencing Platforms

Project Code:

PSLTS-738

Internal Advisor:

Dr. Fahad Maqbool

Project Manager:

Dr. Muhammad Ilyas

Project Team:

i. Muhammad Hassan Shahbaz
ii. Behlol Abbas

Submission Date:

15 Dec 2025

Supervisor's Signatures

Document Information

Category	Information
Customer	UOS – Department of Computer Science
Project	SignLink
Document	Software Design Specification
Document Version	1.0
Identifier	PSLTS-0738
Status	Draft
Author(s)	Muhammad Hassan Shehbaz Behlol Abbas
Approver(s)	Dr. Muhammad Ilyas
Issue Date	Sept. 15, 2025
Document Location	https://github.com/devHassan007/FYP-SLTS
Distribution	1. Advisor 2. PM 3. Project Office

Definition of Terms, Acronyms and Abbreviations

Term	Description
LSTM	Long-Short Term Memory
PSL	Pakistan Sign Language
TTS	Text To Speech
SDK	Software Development Kit
ASL	American Sign Language
BSL	British Sign Language
API	Application Programming Interface
SLR	Sign Language Recognition
VSL	View Sign Language
NLP	Natural Language Processing
GPU	Graphics Processing Unit

Table of Contents

1. Introduction

1.1 Purpose of Document

This Software Design Document (SDD) outlines the architectural framework, component level design, and interface specifications for the Sign Link software application. It serves as a

detailed technical blueprint that guides developers and engineers throughout the implementation phase, ensuring the system's structure aligns with the requirements defined in the SRS. The document provides clear descriptions of modules, data flows, and design decisions to support effective development, testing, maintenance, and future enhancements. By establishing a well-defined design foundation, the SDD promotes consistency, scalability, and efficient communication among all project stakeholders. **Intended Audience:**

- **Developers and ML Engineers:** To implement system components and AI models according to the design specifications.
- **System Architects and Tech Leads:** To review architectural decisions and ensure design consistency.
- **QA Team:** To develop test plans aligned with the system design.
- **Project Managers:** To understand technical complexity and plan resources.
- **Future Maintainers:** To understand the system for debugging and enhancements.
- **Integration Partners:** To understand system interfaces for potential collaboration.

Design Methodology:

We're using **Object-Oriented Design (OOD)** for this project because it offers modularity, reusability, and maintainability—critical for a complex AI-powered application like Sign Link. OOD principles allow us to separate concerns cleanly (recognition, UI, backend services), reuse code across different sign languages and platforms, and easily extend the system with new features without breaking existing functionality.

1.2 Project Overview

SignLink is an accessibility system designed to enable **real-time Pakistan Sign Language (PSL) recognition and Urdu translation** during video conferencing. It integrates with platforms such as **Zoom and Google Meet**, allowing Deaf and hard-of-hearing users to communicate naturally through PSL while hearing participants receive **Urdu subtitles and synthesized Urdu speech**. This enhances digital accessibility, inclusivity, and equal participation in virtual meetings.

What It Does

- Recognizes **PSL gestures in real time** using skeletal key point extraction and deep learning.
- Translates recognized PSL glosses into **Urdu text** and generates **Urdu speech output**.
- Integrates as a **plug-and-play add-on** with major video conferencing platforms (via SDKs/virtual devices).
- Provides **continuous PSL recognition**, allowing natural conversation flow.
- Displays on-screen **Urdu subtitles** and injects **Urdu audio** into the meeting for all participants.

How It Works

The system follows a multi-layered architecture:

- **Computer Vision Layer**
  Uses Media Pipe to detect and extract hand landmarks from the video stream.
- **Machine Learning Pipeline**
  LSTM or Transformer-based deep learning models process extracted key points to recognize PSL gestures.

  Models are trained using **custom PSL datasets**, **Multi-VSL**, and additional PSL video samples collected for this project.
- **Translation Module**
  Converts recognized PSL gloss sequences into meaningful **Urdu sentences**.
- **Text-to-Speech (TTS) Module**
  Generates **English synthesized audio** for hearing participants.
- **Platform Integration Layer**
  Uses virtual camera and virtual audio output or conferencing APIs to inject subtitles and audio into meetings.
- **User Interface Layer Provides**:
  - system configuration options
  - calibration tools
  - real-time display of Urdu subtitles
  - performance indicators (FPS, confidence, latency)

Why It Matters

SignLink empowers Deaf users by enabling natural PSL communication during online meetings **without depending solely on human interpreters**. It significantly improves accessibility for academic, corporate, and social environments by bridging communication gaps between PSL signers and Urdu-speaking participants. The project also contributes to research in **AI-driven sign language recognition**, helping advance accessibility technologies within Pakistan.

1.3 Scope

This project delivers a prototype for **real-time Pakistan Sign Language (PSL) translation** in controlled indoor environments. The system focuses on accurate PSL gesture recognition, Urdu text generation, and English speech synthesis during live video conferencing sessions. The architecture is modular and prepared for future expansion into more complex PSL grammar, multilingual support, and multi-signer interaction.

In-Scope

- **Recognition of a selected PSL vocabulary**, including commonly used words and short phrases based on a custom PSL dataset.

- **Real-time PSL-to-Urdu translation**, converting recognized glosses into grammatically correct Urdu text.
- **Urdu subtitle generation and Urdu speech synthesis**, with audio injected into the video conferencing platform.
- **Integration with at least one major platform** (starting with Zoom) using SDKs or virtual camera/audio devices.
- **Single-signer recognition** in a controlled, well-lit environment with uncluttered backgrounds.
- **End-to-end latency under 100ms** for recognition and translation.
- **Basic user interface** for configuration, calibration, subtitle settings, and performance monitoring (FPS, confidence, latency).
- **Local on-device processing**, avoiding the need for cloud computation.

Out of Scope

- **Full PSL grammar support**, including detailed facial expressions, body shifts, or advanced non-manual markers beyond selected vocabulary.
- **Recognition of multiple simultaneous signers** in the same video frame.
- **Reverse translation (Urdu speech/text to PSL generation)**.
- **Development of a proprietary video conferencing platform** (system relies on existing platforms like Zoom/Google Meet).
- **Robust operation in poor lighting, occlusions, or cluttered backgrounds** (beyond controlled environments).
- **Mobile, tablet, AR/VR, or wearable device optimization** (prototype is desktop-only).

- **Multilingual output** (only Urdu text and Urdu speech are supported in the prototype).
- **Cloud-based or distributed processing frameworks**, focusing solely on local execution.
- **Enterprise-grade security, encryption, or compliance features**, aside from essential privacy measures.

2. Design Considerations

2.1 Assumptions and Dependencies

2.1.1Design-Specific Assumptions

Hardware Requirements

- Users have webcams capable of **720p-1080p resolution at 30 FPS minimum**.
- **CPU:** Intel i5 (8th Generation) or equivalent, with **8 GB RAM minimum**.
- **GPU:** NVIDIA GTX 1060 or equivalent is **recommended** for real-time inference but not mandatory for prototype usage.

Software Environment

- Target Operating Systems:

    - **Windows 10/11** (primary target)
    - **macOS 10.15+**
    - **Linux Ubuntu 20.04+**
- **Python 3.8+** runtime environment available on the system.
- Video conferencing platform SDKs/APIs (Zoom, Google Meet) remain **stable and accessible**.
- Virtual camera and virtual audio drivers can be installed by the user.

Network Conditions

- Stable internet connection for video conferencing (**minimum 5 Mbps**).
- Core sign recognition and translation processing is performed **locally**, requiring no additional bandwidth beyond normal video calls.

User Behavior Assumptions

- Users position themselves within the camera frame with **hands and upper body clearly visible**.
- Signing occurs within a **1 or half meter distance** from the camera.
- Adequate indoor lighting conditions are available.
- Single signer is present in the frame (multi-signer scenarios are out of scope for the prototype).

2.1.2 Dependencies

External Libraries and Frameworks

- **Media Pipe Holistic (Google)** for hand, pose, and facial landmark extraction.
- **TensorFlow / PyTorch** for ML /DL model implementation and inference.
- **OpenCV** for video capture, preprocessing, and frame handling.
- **Zoom SDK / Google Meet APIs** for video conferencing integration.
- **pyttsx3 or Google Text-to-Speech (gTTS)** for Urdu speech synthesis.
-

Datasets

- **Pakistan Sign Language (PSL)** custom dataset.
- **Multi-VSL dataset** for multi-view sign recognition.
- Continuous access to datasets for model retraining and evaluation.

Platform APIs

- Stability of **Zoom Virtual Camera and Audio APIs**.
- Permissions for subtitle overlays within meeting interfaces.

- Continued support for virtual device-based integrations.

Third-Party Services

- Pre-trained **Media Pipe landmark detection models**.
- Text-to-speech engines supporting **Urdu language output**.

2.2 Risks and Volatile Areas

2.2.1High-Risk Areas

Real-Time Performance

- **Risk:** End-to-end latency exceeds the **1s** target, affecting conversational flow.
- **Mitigation:** GPU acceleration, optimized model architectures, frame skipping.
- **Contingency:** Reduce vocabulary size or switch to lightweight recognition models.

Platform API Changes

- **Risk:** Zoom or Google Meet restrict or modify SDK/API access.
- **Mitigation:** Implement a platform-agnostic integration layer.
- **Contingency:** Maintain adapters for multiple conferencing platforms.

Recognition Accuracy

- **Risk:** Accuracy drops below **80%** in real-world usage.
- **Mitigation:** Training with diverse signers and environments.
- **Contingency:** Introduce confidence thresholds and visual alerts for uncertain predictions.

Environmental Variability

- **Risk:** Performance degradation due to lighting, background clutter, or occlusion.
- **Mitigation:** Data augmentation and background preprocessing.
- **Contingency:** Provide user setup guidelines and environment calibration tools.

2.2.2Volatile Areas Requiring Flexible Design

Sign Language Vocabulary Expansion

- Modular recognition models to support vocabulary growth.
- Plugin-based architecture for adding new sign languages (future extension).

Platform Integration

- Adapter pattern for isolating platform-specific logic.

- Clean separation between SignLink core engine and conferencing platforms.

ML Model Updates

- Model versioning and rollback support.
- A/B testing framework for evaluating new model versions.

Output Modalities

- Decoupled translation output pipeline.
- Easy extensibility for:
    - Additional TTS voices
    - Subtitle styling
    - New output languages in future versions

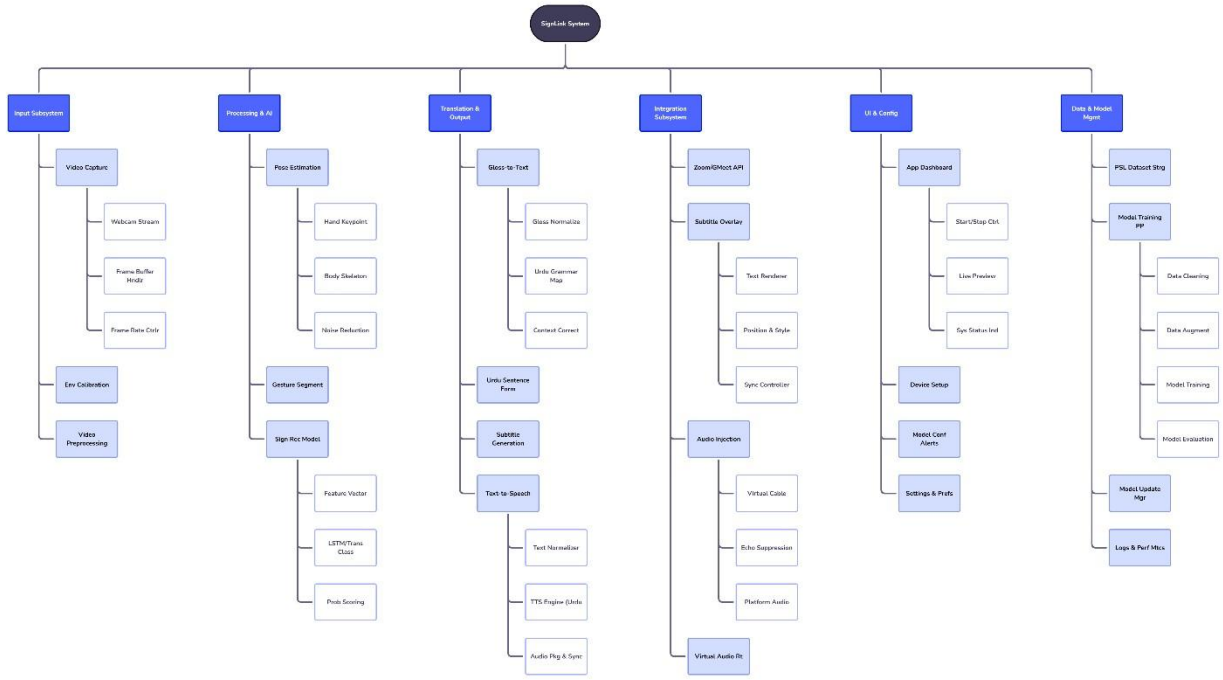
3. System Architecture

3.1 System Level Architecture

High-Level Architecture Overview:

Sign Link follows a **layered architecture pattern** with clear separation of concerns. The system is decomposed into five primary subsystems that interact through well-defined interfaces.

System Decomposition:



Element Relationships:

- **UI Layer ↔ Integration Layer:** Configuration data and status updates
- **Integration Layer ↔ Video Input Manager:** Raw video frames
- **Video Input Manager → CV Processing Layer:** Processed frames
- **CV Processing Layer → ML Recognition Layer:** Keypoint sequences

- **ML Recognition Layer → Translation Layer:** Recognized sign IDs
- **Translation Layer → Output Manager:** Text and audio data

- **Output Manager → Integration Layer:** Formatted subtitles and synthesized speech

External System Interfaces:

- **Video Conferencing Platforms:** Bidirectional communication through SDKs
- **Operating System:** Virtual camera/audio device drivers
- **File System:** Model weights, configuration files, log data
- **User Input Devices:** Webcam, microphone (for calibration)

Physical Deployment:

- **Primary Execution:** User's local machine (desktop/laptop)
- **Model Storage:** Local file system (models loaded into RAM during execution)
- **Video Processing:** Main CPU with optional GPU acceleration
- **Network Communication:** Through existing video conferencing client

Global Design Strategies:

- **Error Handling:** Graceful degradation with user notifications; fallback to silent mode on critical failures
- **Logging:** Comprehensive logging at each layer for debugging and performance monitoring
- **Configuration Management:** Centralized configuration service with validation
- **Resource Management:** Frame buffer pools, model caching, memory-efficient data structures

3.2 Sub-System / Component / Module Level Architecture

3.2.1 User Interface Subsystem

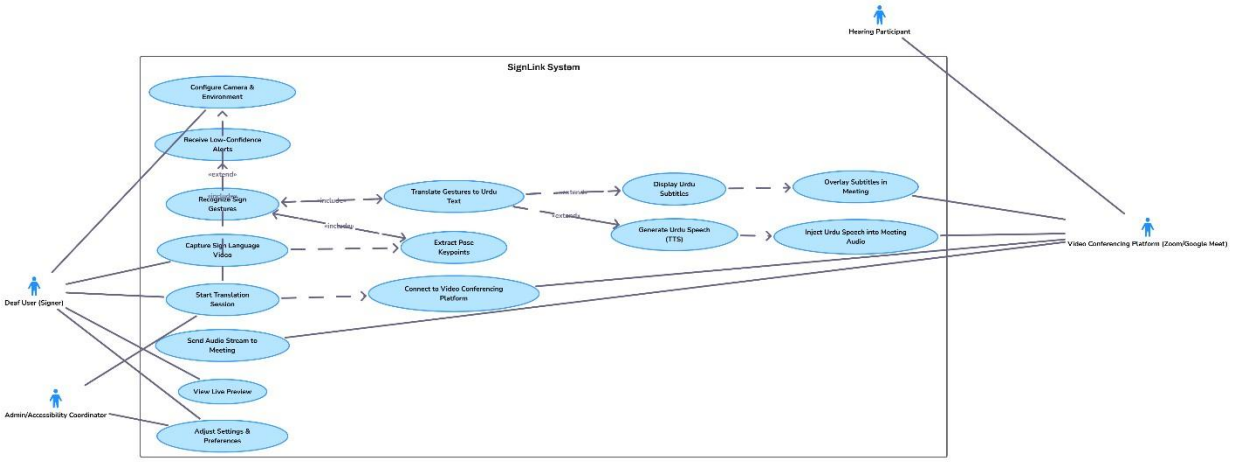
Components:

- **Configuration Panel:** Settings for model selection, platform choice, output preferences
- **Status Monitor:** Real-time display of recognition confidence, FPS, latency
- **Subtitle Renderer:** On-screen text display with customizable positioning and styling
  Notification Service: User alerts for errors, warnings, calibration needs

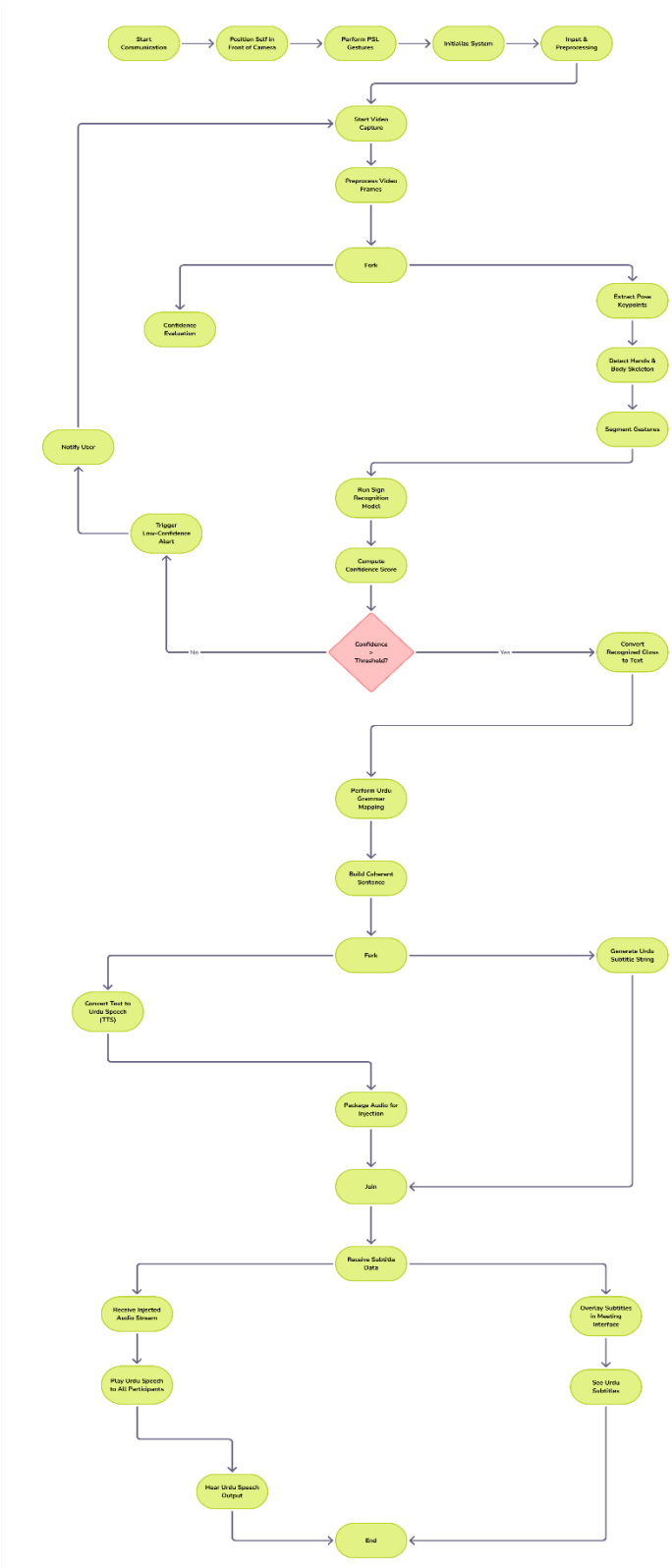
Interactions:

- Receives configuration updates from user
- Sends settings to Platform Integration Layer
- Receives status updates from all lower layers
- Renders subtitles from Translation Layer output

Use Case Diagram:



Communication Flow Diagram:



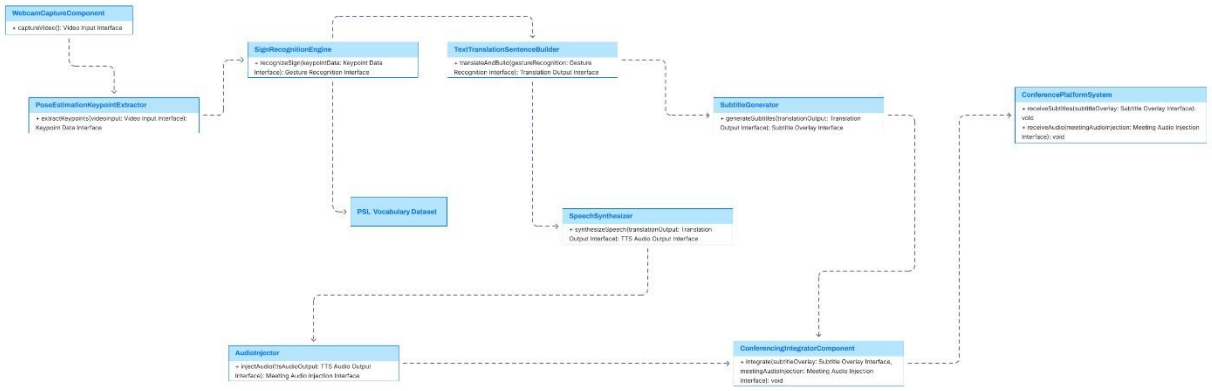
3.2.2 Platform Integration Subsystem

Components:

- **Platform Adapter Factory:** Creates appropriate adapter based on detected platform
- **Generic Platform Adapter:** Fallback using virtual devices
- **Virtual Camera Manager:** Creates/manages virtual camera device
- **Virtual Audio Manager:** Creates/manages virtual audio device
- **Stream Router:** Directs video/audio streams between platform and processing pipeline

Interactions:

- Captures video stream from conferencing platform
- Injects synthesized audio into platform audio stream
- Overlays subtitles on video output
- Provides platform status to UI Layer **Component Diagram :**



3.2.3 Computer Vision Processing Subsystem

Components:

- **Frame Preprocessor:** Resizing, normalization, background blurring
- **Media Pipe Integrator:** Wraps Media Pipe Holistic API
- **Key point Extractor:** Extracts hand (21 points each), pose (33 points), face (468 points) landmarks
- **Sequence Buffer:** Maintains temporal window of key points (e.g., last 30 frames)
- **Data Normalizer:** Normalizes key points to reference frame and scale

Interactions:

- Receives raw frames from Platform Integration
- Outputs normalized key point sequences to ML Layer
- Sends processing metrics to UI Status Monitor

3.2.4 Machine Learning Recognition Subsystem

Components:

- **Model Loader:** Loads and caches trained models
- **Feature Encoder:** Converts key point sequences to model input format
- **BiLSTM Inference Engine:** Processes temporal sequences for sign recognition
- **Transformer Inference Engine (Alternative):** Attention-based recognition
- **Confidence Estimator:** Calculates prediction confidence scores
- **Vocabulary Manager:** Maps model outputs to sign labels
- **Recognition Smoother:** Temporal filtering to reduce jitter

Interactions:

- Receives keypoint sequences from CV Processing Layer
- Outputs recognized sign IDs with confidence scores to Translation Layer
- Provides inference metrics to UI Layer

3.2.5 Translation & Synthesis Subsystem

Components:

- **Sign-to-Text Translator:** Converts sign sequences to URDU
- **Text-to-Speech Engine:** Synthesizes audio from text
- **Subtitle Formatter:** Formats text for on-screen display
- **Output Queue Manager:** Buffers and synchronizes text/audio output

Interactions:

- Receives recognized signs from ML Layer
- Outputs formatted text to Platform Integration for subtitle display
- Outputs synthesized audio to Platform Integration for audio injection

3.3 Sub-Component / Sub-Module Level Architecture

This section provides a detailed breakdown of each major component in the system into its internal sub-modules and sub-components. It also explains how these sub-components interact with one another to enable the complete functionality of the SignLink System. The purpose of this decomposition is to show the internal architecture of each module beyond the high-level system diagram, clarifying responsibilities, data flow, and module boundaries.

3.3.1 User Interface Module — Sub-Components

The **User Interface (UI)** module handles all user-facing interactions. It consists of the following sub-components:

a. Preview Renderer

- Displays real-time camera feed.
- Renders overlays such as status messages, gesture markers, and subtitles.

b. Interaction Controller

- Manages UI actions (Start Capture, Stop Capture, Change Camera).
- Communicates with VideoCapture module via API or local bridge.

c. Status Display Manager

- Shows system states (Processing, Ready, Error).
- Receives status updates from backend components.

Internal Flow:
User Input → Interaction Controller → Backend Trigger

Backend Messages → Status Display Manager → Preview Renderer

3.3.2 Video Capture Module — Sub-Components

The **Video Capture Module** acquires raw frames from the camera device.

a. Device Handler

- Connects to the selected camera hardware (mobile, webcam, IP camera).
- Retrieves frame streams at the configured frame rate.

b. Frame Buffer Manager

- Temporarily stores frames in a buffer for synchronous processing.
- Implements queueing and timestamping.

c. Frame Exporter

- Sends frames to the Preprocessor module.

3.3.3 Preprocessor Module — Sub-Components

The Preprocessor transforms raw frames into a normalized format suitable for ML models.

a. Resolution Scaler

- Resizes frames to the required ML model input dimension (e.g., 256×256).

b. Normalization Engine

- Adjusts lighting, contrast, and removes noise.
- Converts color spaces (RGB → grayscale or model-specific format).

c. Frame Serializer

- Packages the processed frame into a tensor or structured array.
- Forwards it to the Landmarks Extractor.

3.3.4 Landmarks Extractor Module — Sub-Components

The Landmarks Extractor extracts key points (hands etc).

a. Model Loader

- Loads and initializes ML model (Media Pipe, Open Pose, Blaze Pose).

b. Key point Extractor

- Performs inference on processed frames. Produces landmarks + confidence scores.

3.3.5 Gesture Recognizer Module — Sub-Components

The Gesture Recognizer identifies sign-language gestures from a sequence of key points.

a. Sequence Buffer

- Stores key points (temporal sequence).
- Handles sequence segmentation (start/end detection).

b. Classification Engine

- Runs the ML or rule-based classifier (LSTM, Transformer, HMM).
- Outputs gesture label + confidence.

c. Gesture Validator

- Checks confidence threshold.
- Ensures gesture is reliable and not noise.

d. Gesture Event Generator

- Emits recognized gesture events.
- Notifies Translator + stores gesture in database.

3.3.6 Translation Module — Sub-Components

The Translator converts gestures into natural-language sentences.

a. Gloss Mapper

- Maps gesture labels to glossary text.
- Example: SIGN_HELLO → “HELLO”

b. Grammar Rule Engine

- Applies grammar templates and linguistic rules.
- Converts gloss into fluent sentences (e.g., Urdu).

c. Text Output Formatter

- Finalizes translated output.
- Sends result to Subtitle Generator and Text To Speech.

3.3.7 Subtitle Generator Module — Sub-Components

a. Subtitle Builder

- Creates subtitle objects with text and styling.

b. Subtitle Exporter

- Sends them to Platform Integrator for display.

3.3.8 Text-to-Speech (TTS) Module — Sub-Components

a. Voice Model Manager

- Loads the voice model selected for TTS (male/female/accented voices).

b. Speech Synthesizer

- Converts translated text to audio waveform.

c. Audio Packager

- Formats output into MP3/WAV.
- Sends to Platform Integrator .

3.3.9 Platform Integrator Module — Sub-Components

Responsible for delivering subtitles/audio to the target platform.

a. API Connector

- Handles authentication and communication with external platforms.

b. Subtitle Dispatcher

- Sends subtitles in real-time to the selected platform.

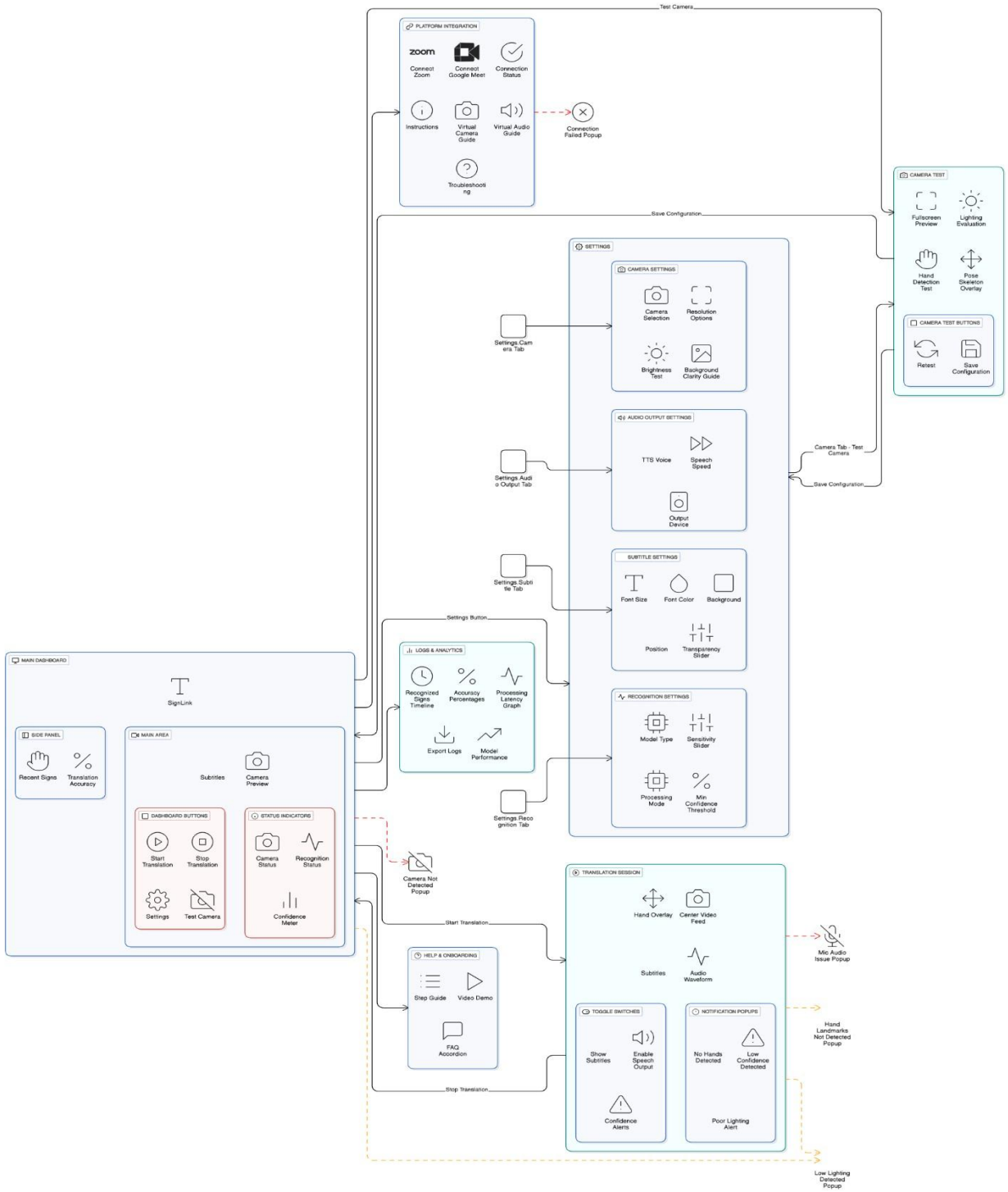
c. Audio Stream Sender

- Streams generated audio to the platform or client.

d. Delivery Monitor

- Confirms successful delivery and logs errors.

Accessibility-Optimized UI Flow :



4. Design Strategies

Strategy 1: Modular and Layered Architecture

The system is organized into clearly defined modules (UI, Business Logic, Data Access), allowing each component to evolve independently.

Reasoning: Supports scalability and easier maintenance.

Trade-offs: Requires careful interface definition between modules.
Impact Areas:

- **Future Extension:** New features can be added without affecting existing modules.
- **System Reuse:** Individual modules can be reused across different parts of the project.

Strategy 2: Consistent User Interface Paradigm

The system follows a uniform UI structure and interaction flow to ensure a smooth user experience.

Reasoning: Reduces user confusion and maintains predictable behavior. **Trade-offs:** Limits experimentation with alternative UI layouts. **Impact Areas:**

- **Future Extension:** New screens maintain the same UI logic.
- **Reuse:** Reusable UI templates and components.

Strategy 3: Centralized Data Management

A centralized database design is used for storing, retrieving, and updating all system data.

Reasoning: Ensures consistency, avoids data duplication, and simplifies auditing.

Trade-offs: The central DB can become a performance bottleneck if not optimized.
Impact Areas:

- **Data Management:** Improves integrity and persistence.
- **Future Enhancement:** Easier migrations and schema updates.

Strategy 4: Concurrency Control & Synchronization

The system maintains controlled access to shared data through safe update mechanisms (e.g., transaction management or locking).

Reasoning: Prevents conflicts when multiple users or processes access the same data.

Trade-offs: Can increase response time during peak usage.

Impact Areas:

- **Concurrency:** Ensures reliable multi-user operations.
- **Data Integrity:** Protects against inconsistent states.

Strategy 5: Reusable Service Components (Service-Oriented Thinking)

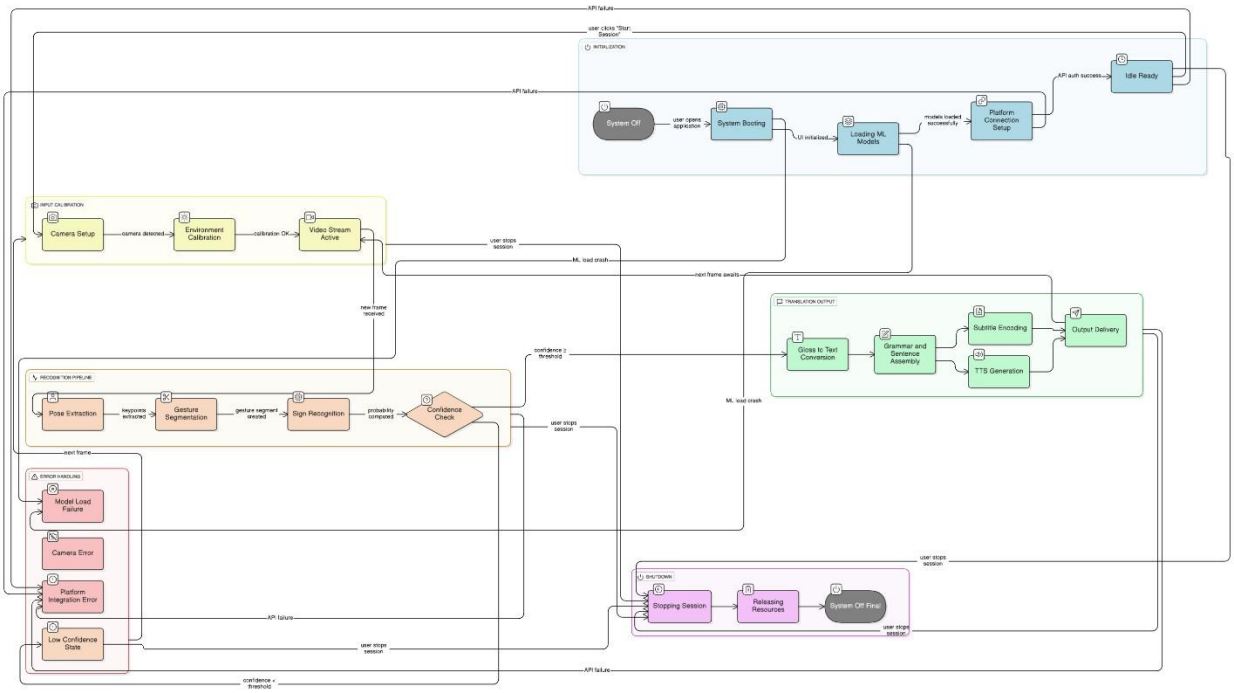
Common operations (authentication, validation, file processing, etc.) are implemented as services accessible across modules.

Reasoning: Minimizes duplication and standardizes logic across the system. **Trade-offs:** Slight overhead in managing shared services. **Impact Areas:**

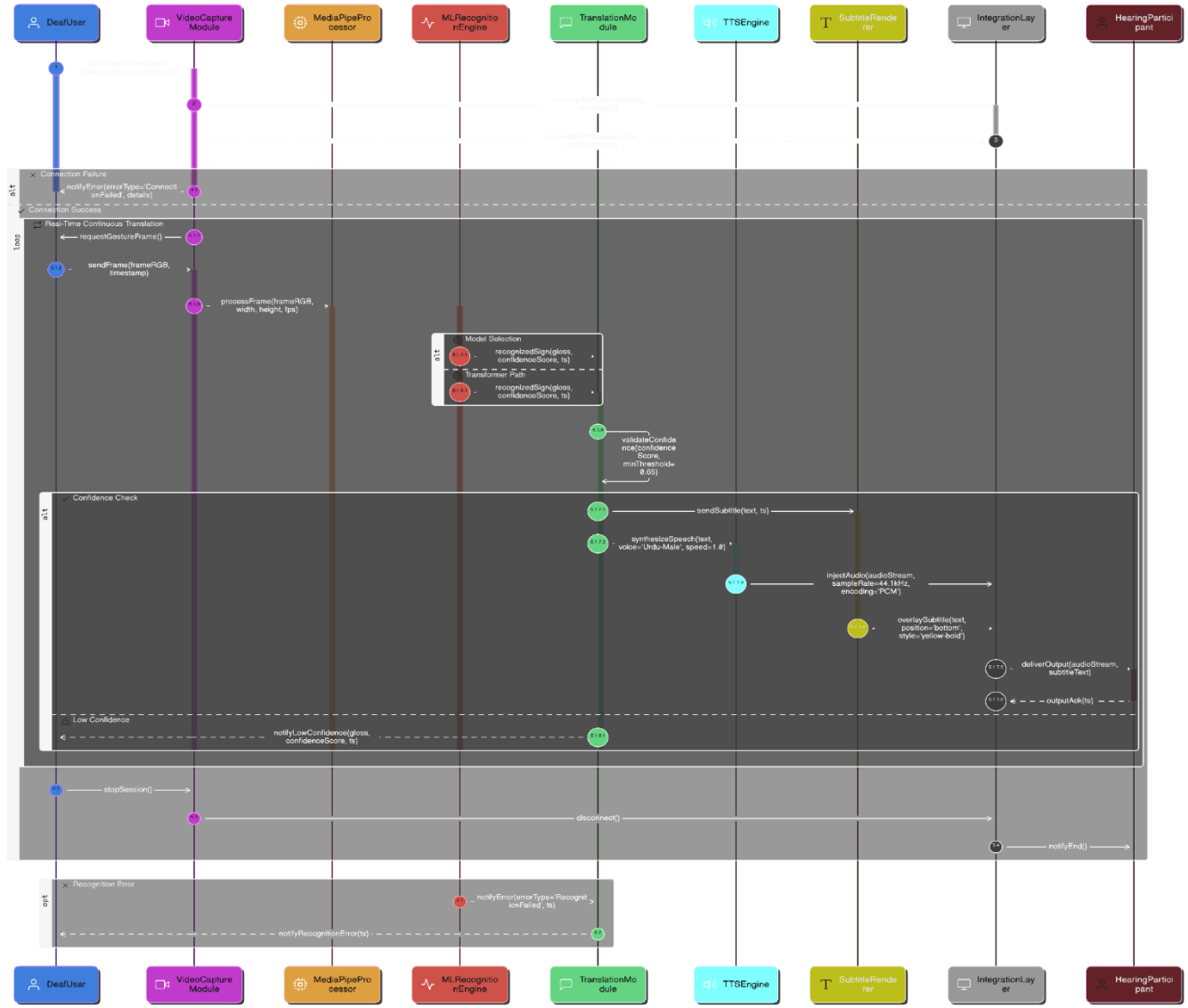
- **System Reuse:** Strong reuse of services throughout the project.
- **Future Enhancements:** Easy integration of new features using existing services.

5. Detailed System Design

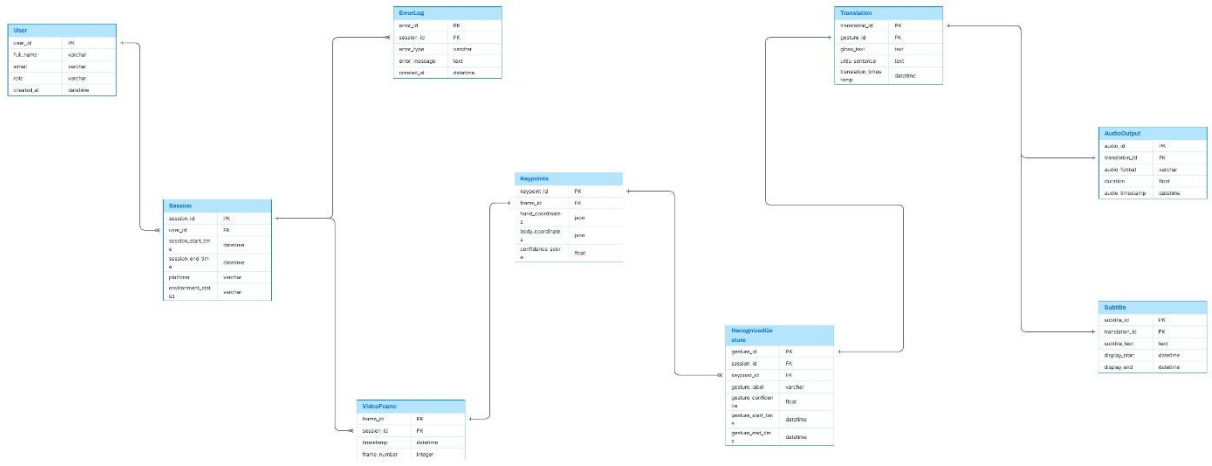
State Change Diagram:



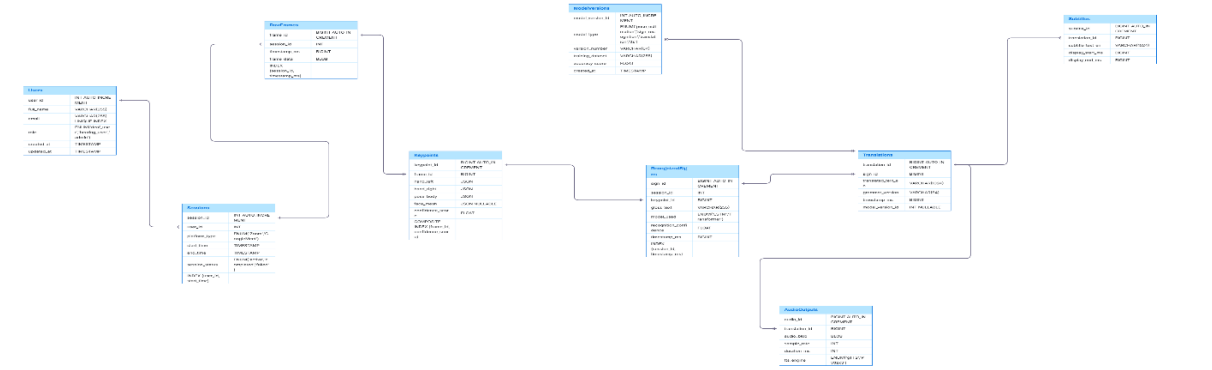
Sequence Diagram:



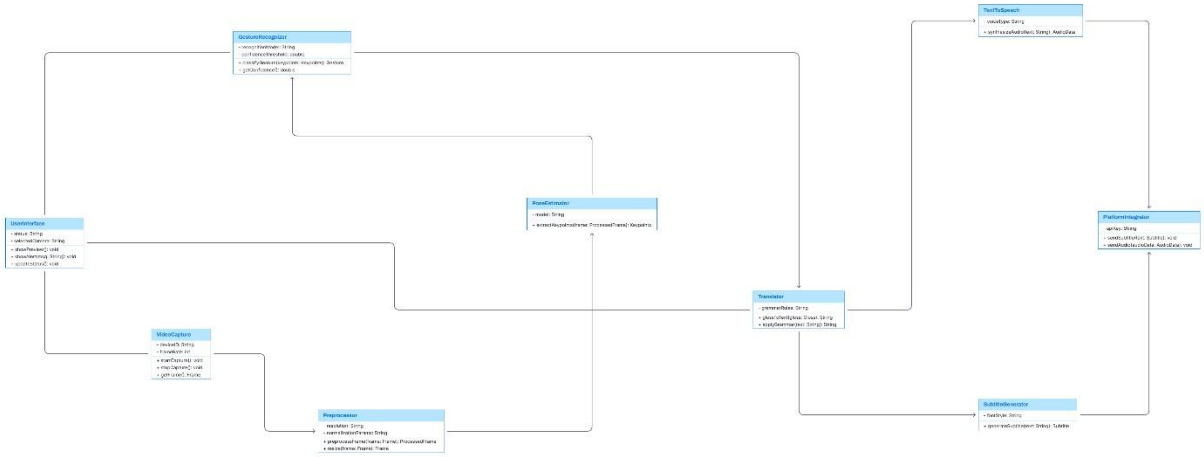
ER Diagram:



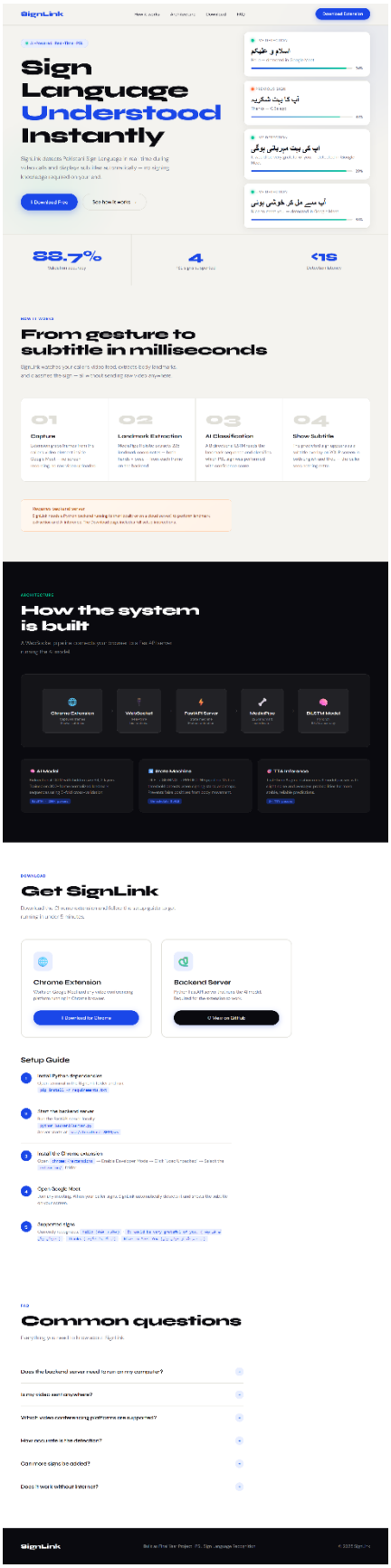
Physical data models:



Class Diagram:



GUI :


SignLink
Download Extension
Sign Language Understood Instantly
Download Free
See how it works
88.7%
4
<1s
From gesture to subtitle in milliseconds
01
Capture
02
Landmark Extraction
03
AI Classification
04
Show Subtitle
How the system is built
Chrome Extension
WebSocket
FastAPI Server
MediaPipe
BiLSTM Model
Get SignLink
Chrome Extension
Backend Server
Download for Chrome
View on GitHub
Setup Guide
Common questions
Does the backend server need to run on my computer?
Is my video sent anywhere?
Which video conferencing platforms are supported?
How accurate is the detection?
Can more signs be added?
Does it work without internet?
SignLink

6. References

Ref. No.	Document Title	Date of Release/ Publication	Document Source
PSLTS-738Proposal	Project Proposal	May 17, 2026	https://github.com/devHassan007/FYP-SLTS
PSLTS-738SRS	SRS	May 17, 2026	https://github.com/devHassan007/FYP-SLTS
PSLTS-738SRS	Diagrams	May 17,2026	https://github.com/behlolabbas/Diagrams
Design Doc	UI	May 17, 2026	https://fyp-pslts.vercel.app/

Poster :


SignLink
FINAL YEAR PROJECT
University of Sargodha
Department of Computer Science
BSCS SELF SS1 (2022-2026)
AI-POWERED TRANSLATION SYSTEM
Pakistani Sign Language
Real-Time Translator
A Chrome extension that detects Pakistani Sign Language during video calls and delivers real-time Urdu subtitles with English speech output — bridging communication barriers instantly.
Chrome Extension
BiLSTM + MediaPipe
88.3% Accuracy
FastAPI + WebSocket
Real-Time Subtitles
KEY FEATURES
Real-Time Detection
Live PSL gesture recognition during Google Meet and other video calls with minimal latency.
Urdu Subtitles
Automatically overlays translated Urdu text as subtitles directly on the video call interface.
English Speech Output
Converts detected sign language into spoken English for a fully accessible communication experience.
225-Landmark Tracking
MediaPipe Holistic extracts 225 body landmarks per frame for precise gesture understanding.
BiLSTM Model
Bidirectional LSTM (hidden=64, 2 layers) trained on normalized landmark sequences for robust accuracy.
Chrome Extension
Manifest V3 extension — load unpacked and it auto-detects signing with zero configuration needed.
SYSTEM ARCHITECTURE
Chrome Extension
Video stream capture
Subtitle overlay UI
Manifest V3
WebSocket
Real-time frame streaming
Low-latency pipe
FastAPI Server
Backend processing
WebSocket handler
Python runtime
MediaPipe
225 landmarks
Holistic model
Pose + Hands
BiLSTM Model
PyTorch inference
Sequence prediction
88.3% accuracy
EXTERNAL SERVICES · TTS Engine · PSL Dataset
MODEL PERFORMANCE
88.3%
TRAINING ACCURACY
± 3.38% variance
100%
TEST ACCURACY
17/17 test cases
225
LANDMARKS / FRAME
MediaPipe Holistic
BiLSTM
MODEL ARCHITECTURE
hidden=64, layers=2
USE CASES
Video Call Accessibility
Deaf/hard-of-hearing users communicate naturally on Google Meet — extension auto-detects signing and overlays subtitles for all participants.
Educational Support
Students using PSL can participate in online classes with real time translation, removing barriers in remote learning environments.
Healthcare Communication
Enables PSL users to communicate with medical professionals during telehealth sessions without a human interpreter.
Inclusive Workplaces
Organizations can conduct inclusive meetings where PSL-speaking employees contribute fully without additional assistive staff.
TECH STACK
AI MODEL
PyTorch BiLSTM
LANDMARK EXTRACTION
MediaPipe Holistic
BACKEND
FastAPI + WebSocket
FRONTEND
Chrome Ext V3 + JS
LANGUAGES
Python · HTML · JS
DATASET
PSL Custom Dataset
PROJECT SUPERVISOR
Dr. Fahad
Dept. of Computer Science
PROJECT MANAGER
Mr. Ilyas
Dept. of Computer Science
GROUP MEMBERS
Hassan Shahbaz
BSCS51F22M07
Bahlol Abbas
BSCS51F22R038
BATCH
BSCS SELF SS1 (2022-2026)
University of Sargodha